

CS 320 Midterm (Version 1)

Total: 100 pts

CS 320 Course Staff

October 16, 2025

Name: _____

BU ID: _____

Instructions

- Please write your name and BU ID.
- This midterm has 5 problems.
- Please read all problems before writing your solutions. You may find some problems easier than others.
- The total duration of the exam is 1 hr 15 mins.
- The typing/semantic rules that you need for each problem are provided **on the same page**. For these rules, sides conditions appear in boxes.

This page is intentionally left blank.

1 Wrong Functions [40 pts]

Recall the typing rule for functions:

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{FUN}$$

Suppose we made a mistake when writing down this typing rule, and forgot to require the type of x in the context of the premise to be the same as the argument type of $\text{fun } x \rightarrow e$ in the conclusion (the difference in the rule is boxed).

$$\frac{\Gamma, \boxed{x : \tau'_1} \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{FUNWRONG}$$

One important property of a programming language is called **preservation**: given any well-typed expression e_0 , if e_0 has type τ_0 , then it must evaluate to a value of the same type τ_0 . In this problem, you will show that a programming language with the rule FUNWRONG violates this property. You'll need (some of) the following typing rules.

$$\begin{array}{c} \boxed{\text{n is an int literal}} \\ \hline \Gamma \vdash n : \text{int} \end{array} \text{INTLIT} \qquad \frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{TRUELIT} \qquad \frac{\boxed{x : \tau \in \Gamma}}{\Gamma \vdash x : \tau} \text{VAR}$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \text{APP}$$

And you'll need (some of) the following semantics rules.

$$\begin{array}{c} \boxed{\text{n is an int literal}} \\ \hline n \Downarrow n \end{array} \text{INTLITEVAL} \qquad \frac{}{\text{true} \Downarrow \top} \text{TRUELITEVAL} \qquad \frac{}{\text{fun } x \rightarrow e \Downarrow (\lambda x. e)} \text{FUN EVAL}$$

$$\frac{e_1 \Downarrow (\lambda x. e) \quad e_2 \Downarrow v_2 \quad \boxed{e' = [v_2/x]e} \quad e' \Downarrow v}{e_1 e_2 \Downarrow v} \text{APPEVAL}$$

Problem 1 (20 pts) Determine an expression e_0 such that $\emptyset \vdash e_0 : \text{bool}$ and $e_0 \Downarrow 42$ according to the above rules (including FUNWRONG). You must provide:

(i) a typing derivation of $\emptyset \vdash e_0 : \text{bool}$ and

(ii) a semantic derivation of $e_0 \Downarrow 42$.

Solution.

Repeated Rules:

$$\begin{array}{c}
\boxed{\text{n is an int literal}} \\
\hline
\Gamma \vdash \text{n} : \text{int} \quad \text{INTLIT}
\end{array}
\qquad
\frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{TRUELIT}
\qquad
\frac{\boxed{x : \tau \in \Gamma}}{\Gamma \vdash x : \tau} \text{VAR}$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 \ e_2 : \tau} \text{APP}
\qquad
\frac{\Gamma, x : \tau'_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{FUNWRONG}$$

$$\frac{\boxed{\text{n is an int literal}}}{\text{n} \Downarrow n} \text{INTLITEVAL}
\qquad
\frac{}{\text{true} \Downarrow \top} \text{TRUELITEVAL}
\qquad
\frac{}{\text{fun } x \rightarrow e \Downarrow (\lambda x. e)} \text{FUN EVAL}$$

$$\frac{e_1 \Downarrow (\lambda x. e) \quad e_2 \Downarrow v_2 \quad \boxed{e' = [v_2/x]e} \quad e' \Downarrow v}{e_1 \ e_2 \Downarrow v} \text{APPEVAL}$$

Solution. (Contd.)

Another important property of a programming language is called **progress**: given any well-typed expression e_0 , it must evaluate to a value. Formally, if $\emptyset \vdash e_0 : \tau_0$, then there must exist a value v_0 such that $e_0 \Downarrow v_0$. In this problem, you will show that a programming language with the rule FUNWRONG violates this property as well. You'll need (some of) the following typing rules.

$$\begin{array}{c}
\boxed{x : \tau \in \Gamma} \text{VAR} \quad \frac{\Gamma, x : \tau'_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{FUNWRONG} \quad \frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \text{APP} \\
\\
\frac{\Gamma \vdash e : \text{bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 : \tau} \text{IF} \quad \frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ADDINT} \\
\\
\frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{TRUELIT} \quad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{FALSELIT} \quad \frac{\boxed{n \text{ is an int literal}}}{\Gamma \vdash n : \text{int}} \text{INTLIT}
\end{array}$$

You'll also need (some of) the following semantics rules.

$$\begin{array}{c}
\frac{}{\text{fun } x \rightarrow e \Downarrow (\lambda x. e)} \text{FUN EVAL} \quad \frac{e_1 \Downarrow (\lambda x. e) \quad e_2 \Downarrow v_2 \quad \boxed{e' = [v_2/x]e} \quad e' \Downarrow v}{e_1 e_2 \Downarrow v} \text{APPEVAL} \\
\\
\frac{e \Downarrow \top \quad e_1 \Downarrow v_1}{\text{if } e \text{ then } e_1 \text{ else } e_2 \Downarrow v_1} \text{IFTRUE EVAL} \quad \frac{e \Downarrow \perp \quad e_2 \Downarrow v_2}{\text{if } e \text{ then } e_1 \text{ else } e_2 \Downarrow v_2} \text{IFFALSE EVAL} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad \boxed{v = v_1 + v_2}}{e_1 + e_2 \Downarrow v} \text{ADDINTEVAL} \quad \frac{\boxed{n \text{ is an int literal}}}{n \Downarrow n} \text{INTLITEVAL} \\
\\
\frac{}{\text{false} \Downarrow \perp} \text{FALSELITEVAL} \quad \frac{}{\text{true} \Downarrow \top} \text{TRUELITEVAL}
\end{array}$$

Problem 2 (20 pts) Determine a type τ_0 and a **well-typed** expression e_0 of type τ_0 such that there does not exist a value v_0 with a valid derivation of $e_0 \Downarrow v_0$. Additionally, provide

(i) a typing derivation of $\emptyset \vdash e_0 : \tau_0$ and

(ii) a brief English explanation of why your choice of e_0 does not evaluate to a value.

Solution.

Repeated Rules:

$$\begin{array}{c}
\frac{\boxed{x : \tau \in \Gamma}}{\Gamma \vdash x : \tau} \text{VAR} \qquad \frac{\Gamma, x : \tau'_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x \rightarrow e : \tau_1 \rightarrow \tau_2} \text{FUNWRONG} \qquad \frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \text{APP} \\
\\
\frac{\Gamma \vdash e : \text{bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 : \tau} \text{IF} \qquad \frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}} \text{ADDINT} \\
\\
\frac{}{\Gamma \vdash \text{true} : \text{bool}} \text{TRUELIT} \qquad \frac{}{\Gamma \vdash \text{false} : \text{bool}} \text{FALSELIT} \qquad \frac{\boxed{n \text{ is an int literal}}}{\Gamma \vdash n : \text{int}} \text{INTLIT} \\
\\
\frac{}{\text{fun } x \rightarrow e \Downarrow (\lambda x. e)} \text{FUN EVAL} \qquad \frac{e_1 \Downarrow (\lambda x. e) \quad e_2 \Downarrow v_2 \quad \boxed{e' = [v_2/x]e} \quad e' \Downarrow v}{e_1 e_2 \Downarrow v} \text{APPEVAL} \\
\\
\frac{e \Downarrow \top \quad e_1 \Downarrow v_1}{\text{if } e \text{ then } e_1 \text{ else } e_2 \Downarrow v_1} \text{IFTRUE EVAL} \qquad \frac{e \Downarrow \perp \quad e_2 \Downarrow v_2}{\text{if } e \text{ then } e_1 \text{ else } e_2 \Downarrow v_2} \text{IFFALSE EVAL} \\
\\
\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2 \quad \boxed{v = v_1 + v_2}}{e_1 + e_2 \Downarrow v} \text{ADDINTEVAL} \qquad \frac{\boxed{n \text{ is an int literal}}}{n \Downarrow n} \text{INTLITEVAL} \\
\\
\frac{}{\text{false} \Downarrow \perp} \text{FALSELITEVAL} \qquad \frac{}{\text{true} \Downarrow \top} \text{TRUELITEVAL}
\end{array}$$

Solution. (Contd.)

2 OCaml Programming [40 pts]

Recall rose trees from Assignment 5:

```
type 'a rtree = Node of 'a * 'a rtree list
```

For the following problem, you can use arithmetic operators from the standard library and any of the following higher-order functions:

```
val List.map : ('a -> 'b) -> 'a list -> 'b list
val List.filter : ('a -> bool) -> 'a list -> 'a list
val List.fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
val List.fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
```

Problem 3 (20 pts) *Implement the function*

```
val sum : int rtree -> int
```

so that `sum t` *is the sum of all the elements at the nodes of* `t`. *For example, the following assertion holds:*

```
let _ = assert (sum (Node 1 [Node 2 []; Node 3 []]) = 6)
```

Solution.

Problem 4 (20 pts) *Determine the smallest type `task` such that the following function is well-typed.*

```
let output (t : task) (x : int) =  
  match t with  
  | Compute(f) -> f x  
  | Double(l) -> List.map (fun e -> 2 * e) l  
  | Filter(p, l) -> List.filter (p x) l  
  | Append(l) -> List.fold_right (fun a b -> a::b) l.second l.first
```

The following signatures are provided for your benefit:

```
val List.map : ('a -> 'b) -> 'a list -> 'b list  
val List.filter : ('a -> bool) -> 'a list -> 'a list  
val List.fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b  
val List.fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
```

Solution.

3 Message Passing [20 pts]

Suppose we introduce a new construct to our language for sending and receiving messages on channels. We introduce three new syntactic constructs:

- ‘let $(tx, rx) = \text{new_channel}$ in e ’: create a new channel with two ends: the variable tx is the sender’s end, which can be used to send messages on the channel, and the variable rx is the receiver’s end, which can be used to receive messages on the channel.
- ‘send e_1 on tx in e ’: evaluate expression e_1 and send its value on tx .
- ‘let $x = \text{recv } rx$ in e ’: receive a value on rx and bind it to variable x .

We introduce two types: τ sender and τ receiver for the sender’s and receiver’s end of the channel respectively. We also provide the typing rules for these constructs:

$$\frac{\Gamma, tx : \tau_1 \text{ sender}, rx : \tau_1 \text{ receiver} \vdash e : \tau_2}{\Gamma \vdash \text{let } (tx, rx) = \text{new_channel in } e : \tau_2} \text{NEWCHAN}$$

$$\frac{\Gamma \vdash tx : \tau_1 \text{ sender} \quad \Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e : \tau_2}{\Gamma \vdash \text{send } e_1 \text{ on } tx \text{ in } e : \tau_2} \text{SEND} \quad \frac{\Gamma \vdash rx : \tau_1 \text{ receiver} \quad \Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{let } x = \text{recv } rx \text{ in } e : \tau_2} \text{RECV}$$

You’ll need the following typing rules.

$$\frac{\boxed{n \text{ is an int literal}}}{\Gamma \vdash n : \text{int}} \text{INTLIT} \quad \frac{\boxed{x : \tau \in \Gamma}}{\Gamma \vdash x : \tau} \text{VAR}$$

Problem 5 (20 pts) Based on these rules, provide a derivation for the following program having type `int` under the empty context $\emptyset$. You should use Δ as shorthand for the context $\{tx : \text{int sender}, rx : \text{int receiver}\}$. Otherwise, you should use no shorthand in your derivation.

```
let (tx, rx) = new_channel in
send 5 on tx in
let x = recv rx in
x
```

Solution.

Repeated Rules:

$$\begin{array}{c}
\boxed{\boxed{\text{n is an int literal}}} \\
\Gamma \vdash \text{n} : \text{int}
\end{array} \text{INTLIT} \qquad
\begin{array}{c}
\boxed{x : \tau \in \Gamma} \\
\Gamma \vdash x : \tau
\end{array} \text{VAR} \qquad
\frac{\Gamma, tx : \tau_1 \text{ sender}, rx : \tau_1 \text{ receiver} \vdash e : \tau_2}{\Gamma \vdash \text{let } (tx, rx) = \text{new_channel in } e : \tau_2} \text{NEWCHAN}$$

$$\frac{\Gamma \vdash tx : \tau_1 \text{ sender} \quad \Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e : \tau_2}{\Gamma \vdash \text{send } e_1 \text{ on } tx \text{ in } e : \tau_2} \text{SEND} \qquad
\frac{\Gamma \vdash rx : \tau_1 \text{ receiver} \quad \Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{let } x = \text{recv } rx \text{ in } e : \tau_2} \text{RECV}$$

Solution. (Contd.)